don’t want Spring Security to reply with an HTTP `302` response redirecting to Keycloak. Instead, you want it to return a response with an error status like HTTP `401 Unauthorized`.

The main reason for not using redirects with SPAs is that you would run into Cross-Origin Request Sharing (CORS) issues. Consider the scenario where an SPA is served from https://client.polarbookshop.com and makes HTTP calls through AJAX to a backend at https://server.polarbookshop.com. The communication is blocked because the two URLs don’t have the same origin (the same protocol, domain, and port). That’s the standard same-origin policy enforced by all web browsers.

CORS is a mechanism for allowing a server to accept HTTP calls through AJAX from a browser-based client like an SPA, even if the two have different origins. In Polar Bookshop, we serve the Angular frontend via the gateway implemented in Edge Service (same origin). Therefore, there aren’t any CORS issues between these two components. However, suppose Spring Security is configured to reply to an unauthenticated AJAX call with a redirect to Keycloak (having a different origin). In that case, the request will be blocked because redirects to different origins are not permitted during AJAX requests.

NOTE To learn more about CORS in Spring Security, you can check out chapter 10 of *Spring Security in Action* by Laurenţiu Spilcă (Manning, 2020), where the subject is explained in great detail. For a comprehensive explanation of CORS, see *CORS in Action* by Monsur Hossain (Manning, 2014).

When changing the Spring Security configuration to reply with an HTTP `401` response to unauthenticated requests, it’s up to the SPA to handle the error and call the backend to initiate the authentication flow. Redirects are only a problem during AJAX requests. The crucial part here is that the call to the backend to start the user authentication is not an AJAX request sent by